

Creating Testing Projects in ASP.NET Core

Testing is an important part of software development. In ASP.NET Core applications, testing helps developers verify that the application works correctly and identifies bugs before the software is released. Visual Studio 2019 provides built-in support for creating and running test projects.

The most commonly used testing frameworks in ASP.NET Core are:

1. **MSTest** – Microsoft's testing framework.
2. **xUnit** – The most popular testing framework for ASP.NET Core applications.
3. **NUnit** – Another widely used unit testing framework.

What is a Test Project?

A **Test Project** is a separate project that contains test methods used to verify the functionality of the main application.

Benefits of Test Projects

- Detect bugs early.
- Improve code quality.
- Simplify maintenance.
- Increase reliability.
- Support continuous integration and deployment.

Types of Testing in ASP.NET Core

1. Unit Testing

Tests individual methods or classes.

Example: Testing an Add() method in a Calculator class.

2. Integration Testing

Tests how different components work together.

Example: Testing database connectivity.

3. Functional Testing

Tests the complete functionality of the application from the user's perspective.

Example: Testing the login process.

Step 1: Create an ASP.NET Core Project

Open Visual Studio 2019

1. Click **Create a new project**.
2. Select **ASP.NET Core Web Application**.
3. Click **Next**.

Configure Project

- Project Name: StudentManagement
- Location: Choose desired folder.
- Click **Create**.

Select Template

Choose:

ASP.NET Core Web Application (Model-View-Controller)

Click **Create**.

The ASP.NET Core project is now ready.

Step 2: Add a Test Project

In Solution Explorer

Right-click on the solution.

Select:

Add → New Project

Search for Test Project

In the search box type:

xUnit Test Project (.NET Core)

Select it and click **Next**.

Configure Test Project

Project Name:

StudentManagement.Tests

Click **Create**.

The solution will now contain:

StudentManagement

StudentManagement.Tests

Step 3: Add Reference to Main Project

The test project must access classes from the main project.

Steps

Right-click:

StudentManagement.Tests

Select:

Add → Project Reference

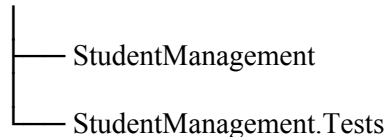
Check:

StudentManagement

Click **OK**.

Solution Structure

Solution



Step 4: Install Required NuGet Packages

Right-click:

StudentManagement.Tests

Select:

Manage NuGet Packages

Install:

xUnit

xunit

xUnit Runner

xunit.runner.visualstudio

Microsoft Test SDK

Microsoft.NET.Test.Sdk

Step 5: Create a Class to Test

Create a class in the main project.

Calculator.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
```

```
namespace StudentManagement
{
    public class Calculator
    {
        public int Add(int a, int b)
        {
            return a + b;
        }
        public int Multiply(int a, int b)
        {
            return a * b;
        }
    }
}
```

Step 6: Create Unit Test Class

Inside:

StudentManagement.Tests

Create:

CalculatorTests.cs

```
using System;
using System.Collections.Generic;
using System.Text;
using Xunit;
using StudentManagement;
using StudentManagement.Models;

namespace StudentManagementTests
{
    public class CalculatorTests
    {
        [Fact]
        public void Add_TwoNumbers_ReturnsSum()
        {
            // Arrange
            Calculator c = new StudentManagement.Calculator();
        }
    }
}
```

```

        // Act
        int result = c.Add(10, 20);
        // Assert
        Assert.Equal(30, result);
    }
}

```

Understanding the Test Method

Arrange

Create objects and prepare data.

Calculator c = new Calculator();

Act

Call the method being tested.

int result = c.Add(10,20);

Assert

Verify the result.

Assert.Equal(30, result);

Step 7: Run the Tests

Open:

Test → Test Explorer

The Test Explorer window appears.

Click:

Run All

Output:

Passed: 1

Failed: 0

Skipped: 0

Green check marks indicate successful tests.

Example: Testing Multiply Method

[Fact]

```

public void Multiply_TwoNumbers_ReturnsProduct()
{

```

```

    Calculator c = new Calculator();

```

```

    int result = c.Multiply(5, 4);

```

```

    Assert.Equal(20, result);
}

```

Running Individual Tests

In Test Explorer:

1. Right-click the test.
2. Select:

Run

or

Debug

Example of Failed Test

[Fact]

```

public void Add_Test()

```

```

{
    Calculator c = new Calculator();

    int result = c.Add(10, 20);

    Assert.Equal(50, result);
}

```

Result:

Failed

Expected: 50

Actual: 30

This helps identify errors quickly.

Using Multiple Test Cases

[Theory]

[InlineData(10,20,30)]

[InlineData(5,5,10)]

[InlineData(100,50,150)]

```

public void Add_TestCases(int a,
                          int b,
                          int expected)
{
    Calculator c = new Calculator();

```

```

    int result = c.Add(a, b);

```

```

    Assert.Equal(expected, result);
}

```

Testing String Methods Example

Main Project

```

public class Student
{
    public string GetName()
    {
        return "Rahul";
    }
}

```

Test Project

[Fact]

```

public void GetName_ReturnsStudentName()
{
    Student s = new Student();

    string result = s.GetName();

    Assert.Equal("Rahul", result);
}

```

Reference video:

<https://www.youtube.com/watch?v=rW6LvPP4VvA>